Reviewing claims against code and evidence

Jeremy H. Carroll

2026-10-02 · Version 0.1.0 · Review protocol · Author-review revision

Code baseline: `52cb5457f6c485ba548cfd292f6012a941ed22cf`.

Purpose

This protocol helps a reviewer decide what a technical claim is supported by. It is a companion to the Proposal Fidelity Protocol (PFP) papers, operator guide, and curriculum guide. "Adversarial" means deliberately testing plausible ways the claim could fail. It does not require hostile language, confidence theater, or agreement among multiple models.

The unit of review is a claim together with its inputs, implementation, and evidence. A successful check supports the property it checked under its assumptions. A failed check or missing prerequisite is useful information to preserve. This document provides a review procedure and worked examples. The only recorded model run it uses is a retained ten-trial trace, which Section 4.1 recounts as an example of reading a reported catch rate. Appendix A applies the procedure to one gate in this repository, the terminal gate of the seven-act Woland engine, with four defect families represented by five probe records from a historical replay on the audited code. It reports no measured psychological effect, search-escape rate, or general automated truth detector.

1. Make the claim small enough to inspect

Replace a broad sentence such as "the system verifies repairs" with a concrete statement:

> For this declared triangle, a replacement proposal that increases the checked quotient minimum is refused, and the active claims remain unchanged.

The reviewer can now identify the graph, original claims, replacement, score definition, relevant branch, and expected stored state. If one is missing, ask the author to supply it before evaluating the broad claim.

An **assumption** is a condition required by the statement. A **witness** is supplied data used to establish a particular property, such as a candidate potential or dual vector. A **receipt** is an execution record. An **open obligation** is a specific unanswered question needed for a stronger conclusion.

Keep a stable source revision and exact artifact paths. When a source file changes, attach a new observation to the new bytes rather than silently reusing the old result.

2. Inspect the connection between language and behavior

For each important term, make a three-column note:

Name in the claim	Property intended	Operation actually performed
"exact repair"	Changed claims satisfy the declared graph relation	A native result is checked under the selected graph inputs

Name in the claim	Property intended	Operation actually performed
"notebook grading"	A learner's answer is assessed	Perhaps only a supplied `execution_status` string is counted
"independent verifier"	Errors can be detected without trusting the proposal	Perhaps only two parameter-hash fields are compared
"replay"	A recorded result is recomputed from its dependencies	Perhaps only one stored digest is compared with another

The third column comes from inspecting the actual call path. Matching names or comments are insufficient. Conversely, a deliberately small check is useful when its stated property is equally small.

The package's semiotic-audit fixture compares the strings `signified` and `interpretant`. It illustrates mismatched descriptions. It does not inspect downstream code, determine mathematical equivalence, or audit every execution path. Use a reviewer to establish those connections unless a separately validated tool actually performs the work.

3. Work through a concrete false reading

Take the triangle claims $h = (1/3, 1/3, 1/3)$. Their cycle sum is one. Suppose an author says that subtracting endpoint-potential differences "removes the cycle obstruction."

Choose potentials $f = (0, 1/3, 2/3)$. Their edge differences are $df = (1/3, 1/3, -2/3)$, so $h - df = (0, 0, 1)$. The discrepancy has been concentrated, but the cycle sum remains one. Every potential produces edge differences summing to zero, so the objection concerns the proposed mechanism, not an unlucky choice of numbers.

A different operation can replace the claims by df, whose cycle sum is zero. That statement is supportable because the declared input changed. The reviewer should request this distinction in the main explanation, not accept a caution several pages later that contradicts the headline.

The cycle lesson reproduces the first reading with these values, and the operator guide shows both readings on an integer triangle. This package's companion notebook and triangle tests recompute the numbers above with exact fractions.

4. Design a probe that could change the verdict

A **positive control** is a case expected to pass. A **negative control** is a case with a known defect expected to fail. The pair tests whether a checker distinguishes cases, rather than accepting or refusing everything.

A **linear program (LP)** minimizes a linear objective subject to linear constraints. A feasible **primal witness** obeys the original constraints; a feasible **dual witness** obeys the associated lower-bound constraints. Matching objective values certify optimality. In the graph example, dual closure means balanced flow at every vertex; the finite graph paper gives the argument.

For an exact LP certificate, first supply feasible primal and dual witnesses with equal objectives. Then independently corrupt a primal bound, break dual closure, and retain a feasible but non-matching dual objective. The LP lesson supplies these three distinct failure cases. These are source-inspected cases; no fresh replay result is reported here. Equal objective values alone cannot replace feasibility.

For a digest-bound packet, change node content while keeping its old digest key. The codec lesson contains that mutation. Checking that a string has 64 characters would not test the same property.

Record the original and changed input, expected distinction, observed result, and reason the probe targets the claim. A probe passing is evidence for that case; exhaustiveness requires an independently justified finite enumeration or proof.

4.1 Reading a reported catch rate

The same controls apply to a reported detection rate. Paper 001's package retains one paired run in which a proposer model classified small Hubbard-model instances as sign-free or as carrying an open sign obstruction, and an adversary model challenged or accepted each classification. The run covers two frustrated lattices, a five-site ring and a six-site cluster of corner-sharing triangles, each at five interaction strengths, for 10 trials. Its replay receipt reports the proposer as `meta-llama/llama-3.1-8b-instruct` and the adversary as `deepseek/deepseek-chat`. The trace itself records no model identifiers.

Each trial's reference label comes from a program check, not from a model. Six proposals claimed a sign-free certificate and were refused by Paper 001's certificate checker; they are labeled unsound. The other four claimed an open sign obstruction and were not refused at the time; they are labeled sound. The receipt describes these as historical verifier labels, not independent physical ground truth.

Checker label	Adversary challenged	Adversary concurred	Trials
Unsound (refused)	6	0	6
Sound (not refused)	2	2	4
All trials	8	2	10

In these 10 trials the adversary challenged all six unsound proposals and missed none. It also challenged two of the four sound-labeled proposals and passed the other two. In both of those challenges its counter-proposal was the same open-obstruction claim, so it disputed the reasoning rather than the conclusion. Counting the trace field `adversary_caught_am` instead gives 10 of 10, because that field is true in every record, including both concurrences. Its name suggests detection; its values do not measure it.

Four observations limit what the run shows:

- All ten trials ended in the same recorded terminal state, an open sign obstruction, whether the adversary challenged or concurred.
- The six refusals come from the checker. The adversary's challenges agreed with them; the run does not show the adversary stopping anything the checker would have accepted.
- A later replay of the ten recorded proposals through a revised version of the checker, stored in the same receipt, refuses all ten. The four obstruction claims fail because none supplies a

checked obstruction witness. The sound label therefore means only that the checker did not refuse the claim at the time.
- The run says nothing about other models, lattices, or prompts.

The package bundles byte-identical copies of the trace and receipt, with provenance and digests. The test test_recorded_bout_counts.py recounts these cells and checks the retained receipt without a sibling package. The exact recount module also gives challenge fractions of $6/6 = 1$ among refused proposals and $2/4 = 1/2$ among not-refused proposals, refused fraction $6/8 = 3/4$ among challenges, and label agreement $(6 + 2)/10 = 4/5$. These are descriptive fractions against historical checker labels. A denominator of zero would leave the corresponding fraction undefined.

The later replay recorded in the receipt does not pin its checker revision. The current package checks the recorded result; it does not rerun that checker or either model. A reproducible native replay would additionally need the checker source revision, executable, dependencies, and command.

5. Read formal statements at their actual scope

Start with the theorem's definitions, hypotheses, and conclusion. Then identify the bridge from the paper's objects to those formal objects.

This package's `lean4` folder holds copies of two of the files discussed below, together with the two modules that the second file imports. Each copy matches the repository's `lean4` file at the code baseline except for a two-line provenance comment at the top. The retained build records use Lean 4.34.0 with no external libraries. The named-theorem report, dated 2026-09-28, lists 32 theorems: 28 have no axiom dependencies, and four use `propext` and/or `Quot.sound`. The full environment census, dated 2026-10-02, lists 516 declarations from those modules, including private and generated declarations, under the approved set `propext`, `Quot.sound`, and `Classical.choice`. These are different census scopes. This review compares source identity and inspects the retained reports; it does not claim a fresh Lean build.

Level15AuthorityCalculus.lean defines distinct state constructors and formalizes that designated proposal constructors differ from the terminal-authority constructor. It also defines an "independent verifier" by unequal parameter hashes and shows that a state does not meet that definition against itself. These facts do not establish statistical independence of different models or guarantee that a runtime preserves the state partition.

TriggerNotLicense.lean constructs a finite case with an active trigger and zero lift budget. It shows that a trigger can exist without a licensed target under those definitions. It does not preclude independently licensed actions under alternate assumptions.

The repository's terminal-routing source `TerminalizerPolicy` imports a Mathlib module and carries a source warning that its declarations were not typechecked against the named package configuration. It is not copied into this package. Its table should therefore not be advertised here as a freshly compiled termination theorem. A build receipt must name the actual toolchain, dependencies, targets, and result.

These examples make the review rule practical: cite what the formal statement says, and identify any missing implementation or modeling correspondence.

6. State the review outcome without promoting it

Use the following report terms as a declared review convention. They are not a new runtime API.

Review disposition	Meaning
Supported within scope	The inspected evidence supports the stated claim under recorded assumptions
Refused claim	A counterexample, failed required check, or contradiction defeats the stated claim
Open obligation	A particular missing bridge or result prevents the stronger conclusion
Not run	The proposed execution has not been performed in this review

A package may contain all four dispositions on different claims. Report them individually. Forwarding a supported result for another review is a workflow step; it grants no publication, deployment, or domain authority.

The package's toy verdict function maps supplied rational scores and flags to `PASS_TO_MASTER`, `REFUSE`, or `OPEN_RESIDUAL`. It does not compute those scores or establish that the flags describe reality. Its supplied-input tests cannot support a claim of comprehensive semantic verification.

7. Preserve enough information for another reviewer

A compact record should contain the claim text, source revision, file path, assumptions, exact input, command, environment or executable pin, observed outcome, and remaining obligation. Include the failing case and any skipped dependency. Identify whether a stored output was only read or was recomputed.

For example, a review request can be recorded before execution:

```
claim_id: triangle-replacement-01
source_revision: 52cb5457f6c485ba548cfd292f6012a941ed22cf
original_claims: ["1", "1", "1"]
proposed_claims: ["5", "0", "0"]
expected_status: DefectIncrease
run_status: NOT_RUN
next_check: Run the focused native-backed session test and inspect stored state
decision_scope: review-only
```

This is a template, not an execution receipt. Replace the pending fields only with observed evidence. A digest binds a declared byte sequence; authenticating a producer and evaluating content are separate tasks. When claiming canonical JSON, test the actual requirements of RFC 8785, including primitive serialization, rather than equating key sorting with complete conformance.

8. Handle stalled work explicitly

When several attempts produce no useful change, record the unchanged residual and the attempted move family. Check whether the current representation permits the requested result. The triangle above demonstrates why more potential adjustments cannot change its cycle sum.

A new representation or new allowed move is a proposal that needs its own assumptions and checks. A stagnation trigger can suggest review; it does not license changing protected state or guarantee an escape. The local fixture only checks a supplied stagnation count, a locked-gauge flag, and a positive supplied score. It does not implement an effective search strategy.

Related work and scope

Necula's proof-carrying code provides a primary example of a producer supplying evidence that a host checks against a declared policy. Saltzer and Schroeder's protection principles provide security-design context for explicit permission and mediation. This protocol borrows a review discipline from these works; it does not inherit their technical results.

The useful deliverable is a review whose reader can trace each conclusion to a precise statement, source, probe, or formal obligation. More ambitious claims about automated reasoning effectiveness require measurements or proofs beyond that trace.

Limitations

This is a review procedure, not a measured method; no study of reviewer accuracy or effort stands behind it. Section 4.1 recounts one retained run of ten trials whose labels come from a program check, and it supports no rate for other settings. Appendix A audits one gate at one commit with five constructed probes in four defect families; it shows that each defect exists, not how often it changes a verdict in use. The package fixtures decide only from supplied values. The copied Lean modules prove narrow statements about their own definitions, and the terminal-routing source discussed in Section 5 states that it was not typechecked. This paper assigns no authorization and changes no jurisdiction. It carries no operational authority or institutional standing.

Verification for this revision

The package verifier (`verify_deposit.py`) checks the manifest and the two-way file census, and `tools/rebuild_release.py --check` checks the release archive against the manifests. The package-local v0.1 review record distinguishes fresh package tests, lint, type checking, an ordinary-Python notebook replay, and release checks from retained reports. The notebook replay executes all five code cells in a fresh CPython process and compares sources, positions, outputs, and execution counts; it does not test Jupyter rendering. A fresh pinned Lean 4.34.0 build is not run in this review because that toolchain is unavailable here. The existing release validations and formal reports retain their dates and scope. The Section 4.1 recount and Appendix A record inspection are fresh checks of historical data, not fresh model, native-checker, or gate runs.

The Landau quiz and study guide provide 24 progressively scoped questions and an executable capstone. Their instructor rubric is local practice guidance; no learner study or qualification claim is made.

References

- Necula, G. C. (1997). Proof-Carrying Code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97)*, pp. 106–119. ACM. doi:10.1145/263699.263712.
- Saltzer, J. H., and Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278–1308. doi:10.1109/PROC.1975.9939. Author-hosted copy.
- Rundgren, A., Jordan, B., and Erdtman, S. (2020). JSON Canonicalization Scheme (JCS). RFC 8785, RFC Editor, June 2020. Independent Submission stream, Informational. doi:10.17487/RFC8785.

Appendix A. Where a gate's verdict comes from

This appendix applies Sections 2, 4 and 7 to one gate in this repository: the terminal gate of the seven-act Woland engine (`omega_engine_woland`) at audited revision 30609de2. The gate is meant to authorize a proposal only when exact defect measures of the input are zero: a quotient defect, written Φ_1, and a cycle defect, written Γ_2. Otherwise it should refuse, or record a bounded residual. The review question is the one of Section 2: for each verdict, which operation actually produced it?

Four defect families, represented by five probes (F1, F2a, F2b, F3, F4), answer that question. Each fixes an input, states the status a sound gate must give, and records the status the audited code gave. They were run on the audited commit by a script of this collection, `tools/replay_woland_gate_falsifiers.py`, which extracts the needed packages with `git archive` and calls single act functions. The stub gate runtime has no event sinks or model port; the script extracts files to a temporary directory and its `--write` mode writes the JSON record. The observed values below come from the retained collection record, `verification/woland_gate_falsifiers.json`. A byte-identical copy and its provenance are included here. This review inspects that record; executing the original gate again requires a repository checkout containing the audited commit. File and line references are to the audited commit.

Table A.1. The name of each operation and what it does.

Name in the gate	Property intended	Operation actually performed
"Gauge break" and "carrier lift" (Act IV)	A lifted carrier on which the input's obstruction can be examined	A node is added and the cochain is replaced by zero on every edge (`omega_engine_woland/use_cases/act_helpers.py`, lines 480 to 498); Act II then scores the zeros (`use_cases/bout_service.py`, lines 266 to 277)
"Independent witness" and "countermodel" (Act V)	A critique blocks a candidate when it establishes a defect	A critique blocks when its text contains the word "DEFECT" or "FLAW" (`act_helpers.py`, line 642)
"Evidence references" and "obstruction non-vanishing receipt" (Act VII)	Each reference names stored bytes that support the verdict	With no critique, the reference is the SHA-256 of the run key; the receipt's identifier is the SHA-256 of a label string, and its fields `annihilates_coboundary` and `norm_bounded` are the constant `True` (lines 917 to 932 and 1008 to 1015); every event record's `carrier_hash` is the SHA-256 of the fixed bytes `carrier` (six sites, from line 179)

Name in the gate	Property intended	Operation actually performed
"Novella" and "DDC tape" of a challenge gate	A narrative and a step record of what the run did	Fixed text and fixed steps into which only the verdict, the counts and two numbers are substituted (C012 copy, `governance/woland_gate.py`, lines 69 to 195)

Table A.2. Falsifiers: fixed input, expected status, observed status.

ID	Fixed input	A sound gate gives	The audited gate gave
F1	Triangle with edges (0,1), (1,2), (2,0) and cochain (1, 1, 1), whose cycle sum is 3; default scorer; one candidate; no critique	Not `AUTHORIZED`: no potential changes a cycle sum (Section 3)	`AUTHORIZED`
F2a	Critic reply: "Counterexample: the residual (0,0,3) has cycle sum 3, so the claimed repair is false."	Blocking	Not blocking; the candidate survives
F2b	Critic reply: "No defect found; the certificate checks."	Not blocking	Blocking; the candidate is removed
F3	Act VII with no critique, $\Phi_1 = 1$, $\Gamma_2 = 0$, and a dual witness of 1 on edge (0,1) only	Each reference is the digest of stored evidence; stated witness properties are checked	First reference is the SHA-256 of the run key; the receipt states that the witness annihilates coboundaries and is norm-bounded, although it is not balanced
F4	C012 gate with verdict `REFUSE`, 0 cases, 0 passed, 5 defects	No statement of repair, agreement or budget that the inputs do not support	The novella states "100% concordance across all cases", a respected budget, and "zero duality gap on all 46 judge cases", and names challenge C011; the step record contains an exact coboundary repair with status "repaired"

A.1 Input replacement (F1)

On the triangle, the default scorer reports 3 in place of Φ_1, and $\Gamma_2 = 0$. That scorer returns the ℓ^1 mass of the cochain, not the quotient defect, and it returns $\Gamma_2 = 0$ for every input. Act IV fires because the score is positive and only one candidate exists. It returns a lifted carrier whose cochain is zero on every edge, and Act II on that cochain reports zero for both measures. Act VII then authorizes. The verdict comes from the replaced input, not from the proposal. This is the false

reading of Section 3 in executable form: replacing the claims changes the input, and the gate then certifies the replacement.

Fix: commit a054c8ce1 on branch `claude/mssn-study-corrections`. An Act IV lift now carries the input cochain to the lifted carrier, and a lift that lowers the measured obstruction is refused with `WOLAND_LIFT_LOWERED_OBSTRUCTION`. The default scorer reports that nothing was measured, which cannot authorize, and Act VII requires exact rational zeros. The laundering falsifiers are in `omega_engine_woland/tests/test_honest_gate_falsifiers.py`.

A.2 Keyword adjudication (F2a, F2b)

The critic's reply is a proposal like any other, and Section 2 asks what the gate does with it. It searches the text for two words. A reply that states a counterexample without either word does not block, and a reply that says no defect was found does block. The status of the candidate therefore depends on the critic's wording, not on a check of what the critique claims.

Fix: commit a054c8ce1. A critique is now a typed JSON block, and a critique that cannot be parsed blocks; no keyword decides. The keyword falsifiers are in the same test file and in `omega_engine_woland/tests/test_adversarial_m1_it2_1_challenger.py`.

A.3 Phantom evidence references (F3)

An evidence reference is useful only if another reviewer can retrieve the bytes it names (Section 7). With no critique, the audited gate substitutes the SHA-256 of the run key, which names no stored evidence. The identifier of the obstruction receipt is the SHA-256 of a label built from the run key and the dual value, not the digest of the receipt. The receipt itself asserts that the dual witness annihilates coboundaries and is norm-bounded, but those fields are constants: the witness of F3 has nonzero net flow at vertices 0 and 1, and no weights were supplied to bound it. The event records carry the same `carrier_hash` in every run.

Fix: commit a054c8ce1. The obstruction and deficit receipts are stored with the run under `evidence_receipts`, keyed by the digest of their canonical bytes, and the verdict cites them by that digest. The constant witness fields are removed, and the run's `receipt_id` is computed from its content.

A.4 Templated receipts (F4)

The challenge packages carry their own copies of a gate module that writes a narrative ("novella") and a hash-chained step record for each run. In the C012 copy, both are templates. For a refused run with no cases, the narrative still reports full agreement of the two checkers, a respected budget and models under 70B, and its default summary claims verified repairs on 46 judge cases; the step record lists an exact simplex descent and a repaired coboundary. The copy also names challenge C011, from which it was copied. The chain of digests over these steps is internally consistent (Section 7), so a reader who checks only the chain would accept them. The challenges folder holds 36 files named `governance/woland_gate.py`, and the sentence about full agreement occurs verbatim in four of them.

Fix: commit e6fbc15cf on the same branch. The 36 challenge copies of the gate now report a reference run as `REFERENCE_SELF_CONSISTENT` or `REFERENCE_SELF_INCONSISTENT` with a stated `gate_scope`, and the narrative is a table of the measured counts with no templated claims. The tests are in `omega_engine_projects/mssn_sn_symgam/challenges/harness_tests/test_harness_honesty.py`.

A.5 What the audit supports

The five probes in four defect families show that, at the audited commit, an `AUTHORIZED` status from this gate does not establish that the proposal repaired its input, that a blocking critique established a defect, or that a cited reference names evidence. They do not show how often these defects changed a verdict in recorded runs, which this appendix does not examine. In the corrected gate, authorization requires an eligible candidate and no applicable blockers, together with an `ACCEPTED` checker verdict bound to that candidate by digest. When a carrier is declared, verified exact joint zeros are also required; with no declared carrier, the carrier-zero requirement does not apply. The receipt field `authorization_evidence` records which of the two applied. The verdict therefore depends on a bound checker receipt, the property Section 2 asks a reviewer to locate. Binding identifies the candidate and receipt bytes; it does not by itself establish the checker's correctness, the producer's authenticity, statistical independence, or the truth of a parsed critique. The corrected-code descriptions above are source observations at the named revisions. The earlier review reported 66 and 44 passing tests for the two Woland files; those historical counts are not fresh executions in this v0.1 review.